

Building an `async/await` task library in C++20

Welcome!

- I'm June (She/Her)
- I integrate Epic Online Services into Unreal Engine
- EOS plugin has a lot of asynchronous code
(most EOS SDK calls are async)
- Chaining lots of sequential async calls is painful with callbacks
- I wrote an async task library with C++20 coroutines to make things more maintainable



Agenda

- Callbacks vs `co_await`
- C++ coroutines spec and task library design
- Building out the task library
 - Lifetime control
 - Imposed cancellation
 - Interacting with synchronous code
 - Requested cancellation
 - Async lambdas

Callbacks vs co_await

Callbacks are hard to maintain

1. Function returning on a different frame
 2. Pass a callback in, which gets called when it's complete
- Hard to statically guarantee that the function always calls the callback
 - Lambda callbacks get deeply nested after even a few asynchronous calls

```
FMultiOperation<FUNresolvedFriendEntry, TResolvedFriendEntry>::RunBatched(
    EntriesByType.Value,
    EOS_CONNECT_QUERYEXTERNALACCOUNTMAPPINGS_MAX_ACCOUNT_IDS,
    [WeakThis = GetWeakThis(this), EntriesByType, EOSUser](
        TArray<FUNresolvedFriendEntry> &Batch,
        const std::function<void(TArray<TResolvedFriendEntry> OutBatch)> &OnDone) -> bool {
    if (auto This = PinWeakThis(WeakThis))
    {
        EOS_Connect_QueryExternalAccountMappingsOptions Opts = {};
        Opts.ApiVersion = EOS_CONNECT_QUERYEXTERNALACCOUNTMAPPINGS_API_LATEST;
        Opts.AccountIdType = EntriesByType.Key;
        Opts.LocalUserId = EOSUser->GetProductUserId();
        EOSString_Connect_ExternalAccountId::AllocateToCharListViaAccessor<FUNresolvedFriendEntry>(
            Batch,
            [](const FUNresolvedFriendEntry &Entry) {
                return Entry->ExternalInfo.AccountId;
            },
            Opts.ExternalAccountIdCount,
            Opts.ExternalAccountIds);

        EOSRunOperation<
            EOS_HConnect,
            EOS_Connect_QueryExternalAccountMappingsOptions,
            EOS_Connect_QueryExternalAccountMappingsCallbackInfo>(
                This->EOSConnect,
                &Opts,
                EOS_Connect_QueryExternalAccountMappings,
                [WeakThis = GetWeakThis(This), Opts, Batch, OnDone, EOSUser, EntriesByType](
                    const EOS_Connect_QueryExternalAccountMappingsCallbackInfo *Info) {
                    if (auto This = PinWeakThis(WeakThis))
                    {
                        TArray<TResolvedFriendEntry> Results;

                        auto FreeMem = [Opts]() {
                            EOSString_Connect_ExternalAccountId::FreeFromCharListConst(
                                Opts.ExternalAccountIdCount,
                                Opts.ExternalAccountIds);
                        };

                        if (Info->ResultCode != EOS_EResult::EOS_Success)
                        {
                            // ...
                        }
                    }
                }
            );
    }
}
```

C++20 introduces coroutines with `co_await`

- Async/await just like in C#
- No deep nesting
- Compiler guarantees return
- Coroutines spec is very powerful and can do a lot more than just async tasks
- C++20 gives us coroutines spec
- But leaves building a task library to the developer

```
static TTask<void> VoidCoroutine()
{
    co_await FDelayOnAnotherThread{};

    co_return;
}

static TTask<int> SimpleCoroutine()
{
    co_await VoidCoroutine();

    co_return 5;
}

TTask<int> AsyncMain()
{
    int Value = co_await SimpleCoroutine();

    printf("got value from SimpleCoroutine: %d\n", Value);

    co_return 0;
}
```

Core C++20 coroutines concepts

Two sides of coroutines

Coroutines

Functions that get translated to a state machine:

- ***promise object***
Manipulated from inside the coroutine. This is the thing that specifies how the coroutine will behave.
- ***coroutine handle***
Can be used to get the promise object during suspend/resume.
- ***coroutine state***
Internal dynamically allocated storage.

Awaitables

Things that coroutines can `co_await` on, with the following functions:

- ***await_ready***
Check if `await_suspend` can be skipped (is the result already available?)
- ***await_suspend***
Called to suspend the calling coroutine, with the coroutine handle passed in.
- ***await_resume***
Returns the result of the awaitable.

How does the compiler know a function can be a coroutine?

- Either `promise_type` alias/subtype on the returned type:

```
struct task
{
    struct promise_type
    {
        task get_return_object() { return {}; }
        std::suspend_never initial_suspend() { return {}; }
        std::suspend_never final_suspend() noexcept { return {}; }
        void return_void() {}
        void unhandled_exception() {}
    };
};

task resuming_on_new_thread(std::jthread& out)
{
```

- Or via `coroutine_traits`:

(This is the variant we will be using for our library)

type parameters for `coroutine_traits` is:

- The return type of the function, followed by
- The type of each function argument

```
// Specialise coroutine_traits so the compiler knows the promise type to use.
namespace std
{
    template <typename ReturnType>
    struct coroutine_traits<TTask<ReturnType>>
    {
        using promise_type = ::TCoroutinePromise<ReturnType>;
    };
}
```

Required promise object functions

- ***get_return_object***
Used to construct the actual return value from the coroutine e.g. `TTask<ReturnType>`
- ***initial_suspend***
Called at the start of the coroutine. Can be used to move the coroutine execution to a different thread before anything executes.
- ***final_suspend***
Called at the end of the coroutine.
- ***return_void / return_value***
Called when the `co_return` expression is used, prior to the coroutine being destroyed.
- ***unhandled_exception***
Called on an unhandled exception, `std::current_exception()` can be used to get the exception and store it for rethrowing later.

Optional promise object functions

- ***await_transform***

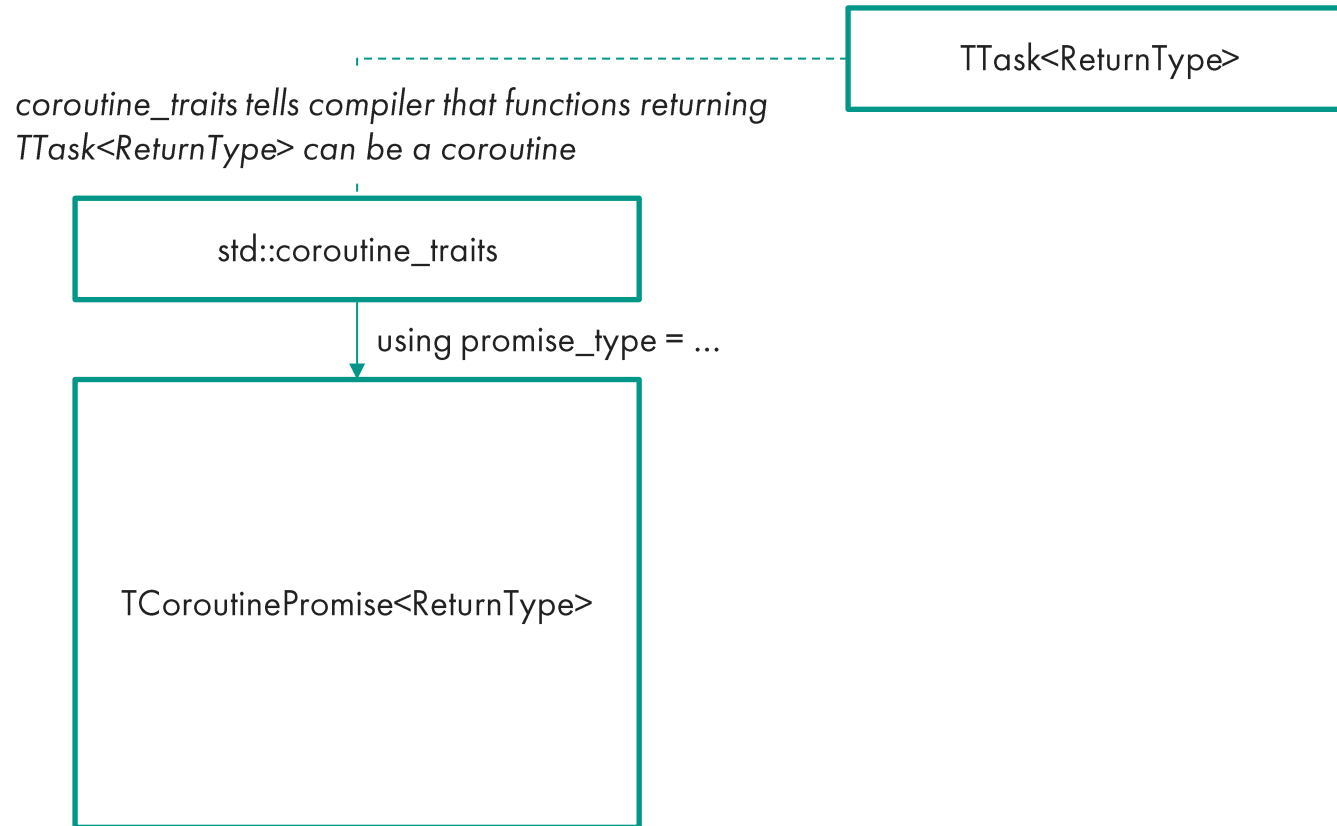
Accepts an awaitable as the first argument, and can return a new awaitable. Useful if you need to intercept `co_await` or wrap awaitables.

- ***yield_value***

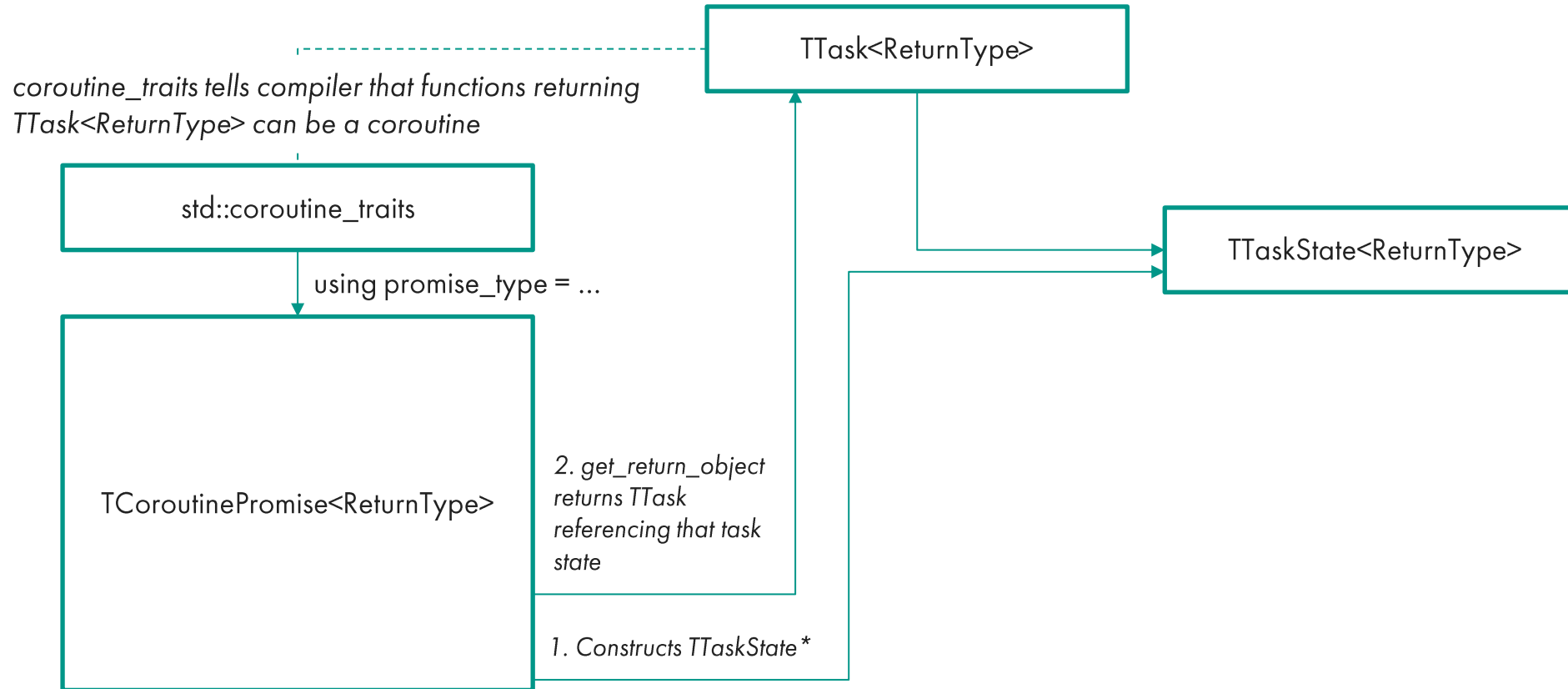
Called when the `co_yield` expression is used. Only useful if you're implementing generators or something like `IAsyncEnumerable`.

Task library design

Type relationships / Target design

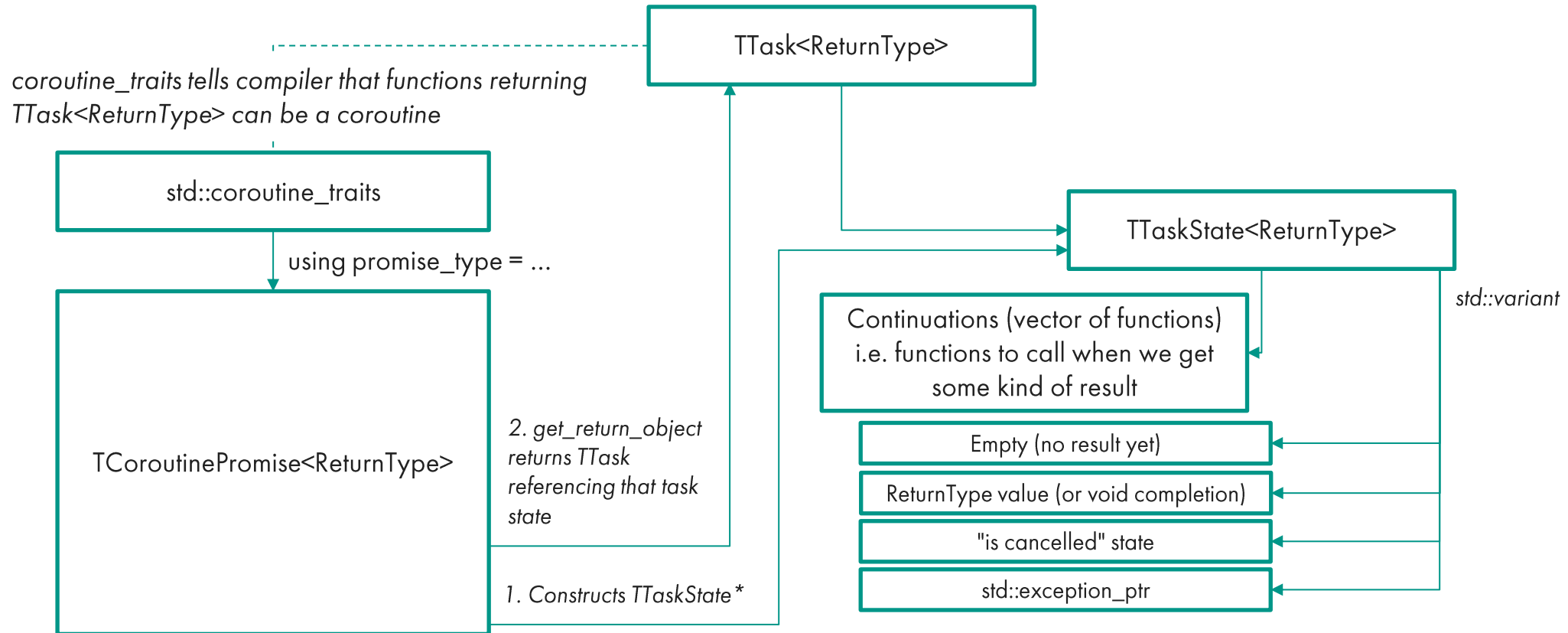


Type relationships / Target design



* Technically you can defer heap allocating the task state in the coroutine promise until the first suspension point, eliminating one heap allocation if the coroutine immediately does `co_return`. For simplicity, we won't show that this talk.

Type relationships / Target design



* Technically you can defer heap allocating the task state in the coroutine promise until the first suspension point, eliminating one heap allocation if the coroutine immediately does `co_return`. For simplicity, we won't show that this talk.

Example: 01_SimpleCoroutine, 02_SimpleAwaitable

- These examples are much more simplified than design outlined (we will get there but I want to start as simple as possible)

Example: 03_TaskAwaitable

- Let's build out more of our design

Example: 04_TaskVoidAwaitable

- Handling void values
- Can't declare a field of type void
- So we need to specialise some of our templates

Lifetime control

What about coroutines on objects?

- So far all our coroutines are static functions
- No 'this' pointer
- Supporting coroutines on objects would be useful
- But how do we make sure that it's valid for a coroutine to resume?
 - i.e. handling an object being free across a suspension point

Propagate lifetime checks to e.g. `std::enable_shared_from_this`

Things we need to do:

1. `TTaskState` -> `TTaskStateBase` + derived variants with "IsValid" overload
 - `TTaskStateStatic`
 - `TTaskStateSharedFromThis`
2. Add "promise_is_valid" to `TCoroutinePromise` and route to state "IsValid"
 - This is how `await_suspend` will check if it's valid to resume the coroutine
3. Update "await_suspend" to use it
4. Update `std::coroutine_traits` to use derived task state based on function signature

Example: 05_SharedFromThisLifetime

Gotcha: the `await_suspend` of the `awaitable` is checking the lifetime

- Remember: `await_suspend` is on the `awaitable`
 - NOT the coroutine who wants to do lifetime checking
- You need to make sure all your `await_suspend` implementations do the lifetime check
- If you are using third party libraries that provide `await_suspend`, be aware they will not do your lifetime checks
- Also: `await_suspend` needs to check the lifetime of the handle you are resuming
 - For `TTask`, NOT the lifetime of it's own `TTaskState`

Detour: Supporting arguments

- None of our coroutines allow arguments at the moment
- To support this, we need to modify `coroutine_traits` further
- However, now it is ambiguous on the first argument:
 - Is this a member function with a `this` pointer of an object type that isn't supported?
 - Or is it a static function with the first argument a pointer to an object with `enable_shared_from_this` that should not be used for lifetime control?
- `ETaskBinding` enumeration to specialize `TTask`
 - Specializing `TTask` allows us to specialize `coroutine_traits`
 - Because coroutines are inferred from the return type only

Intellisense doesn't handle coroutine_traits properly

- Intellisense gives up with our many specialized coroutine_traits definitions
- You'll want to use a guard on `__INTELLISENSE__` to provide much more simple definitions

Example: 06_Arguments

Imposed cancellation

What to do when a coroutine can't resume?

- If the 'this' pointer is no longer valid...
 - `std::terminate` isn't a practical option
 - How can the caller know that they will never get a result?
- Make "cancelled" a possible state for tasks
- Cancellation propagates upwards through `co_await`
 - If you `co_await` a task that cancels, you (as a coroutine) also become cancelled

This is where things start to get somewhat non-trivial

What we need to do:

- TTaskState needs to store its result as a variant (and not optional)
- TTaskState needs a better API for emplacing value or cancellation states, as well as adding continuations
- At the moment we have a separate specialization of TTaskState for void. To avoid code duplication, we're going to move the specialization for void into new templates so we can have one TTaskState declaration.
- Likewise, we also need to reduce code duplication for TCoroutinePromise

Example: 07_ImposedCancellation

- General refactoring
 - CoroutinePromise.h
 - TaskState.h
- Simplifying APIs
 - promise_continue/promise_cancel in TCoroutinePromiseBase
 - Replaces promise_is_valid
 - AddContinuation/AddContinuationFromSuspendingParentCoroutine in TTaskStateBase
 - Replaces public access on the TTaskStateBase continuations

Interacting with synchronous code

Deferred tasks and 'AsCallback'

Deferred tasks (*08_DeferredTasks*)

We need to be able to have functions that return TTask which aren't coroutines themselves

- These should be able to return a "deferred" TTask
- That is, a task that we will explicitly set a value or cancellation into at a later point

Important to allow asynchronous code to await on callback-based APIs

AsCallback (09_AsCallback)

We need to allow callback-based functions to start coroutines and broadcast callbacks when they complete.

AsCallback() function which accepts:

- The TTask to wait for completion
- A lambda to run when the task returns a result
- A lambda to run when the task is cancelled (and no result is available)

Callers of AsCallback can then broadcast their callbacks inside those lambdas as appropriate.

Requested cancellation
Plus 'AsPotentiallyCancelled'

Requested cancellation

We might want to be able to request that coroutines are cancelled at their next `co_await` point, even if their lifetime is still valid and they have a result from the `co_await`'d expression

Comparison:

- Imposed cancellation propagates up the stack from a coroutine with an invalid lifetime to the callers.
- Requested cancellation propagates down the stack from a coroutine into the deepest point being awaited.

Requesting cancellation is not immediate; the coroutine cancels at the next resumption point.

Example: 10_RequestedCancellation

AsPotentiallyCancelled

Need to be able to `co_await` tasks that might be cancelled, and "capture" imposed cancellation to stop it propagating all the way to the root of the async stack.

`AsPotentiallyCancelled` takes a `TTask<ReturnType>` and returns `TTask<std::optional<ReturnType>>`, returning an unset optional if the task was cancelled.

Tip: Users should prefer `AsPotentiallyCancelled` over `AsCallback` if running inside a coroutine, because `AsPotentiallyCancelled` is being `co_await`'d and can propagate requested cancellation (`AsCallback` can not)

Example: 1 1_AsPotentiallyCancelled

Async lambdas

Async lambdas & lifetime control

We'd like to have async support for lambdas

But lambdas with captures pose a tricky problem for lifetime control

Lambdas with captures get transformed into anonymous classes

So the first argument of `coroutine_traits` is the anonymous lambda class, not any 'this' pointer in the capture list

Inferring the lifetime is not possible for async lambdas

Solution: unbound lambdas and "Bind"

Require async lambdas to have a non-constructable "IUnbound" as the first argument, and erase it via a Bind function that specifies the lifetime:

```
auto UnboundAsyncLambda = [] (IUnbound) -> TTask<int, ETaskBinding::Unbound> {
    // ...
    co_return 5;
};

// Can't call UnboundAsyncLambda here because we can't make an 'IUnbound' for first argument.
co_await UnboundAsyncLambda(IUnbound());

auto BoundAsyncLambda = Bind(this, UnboundAsyncLambda);

// Now we can call the lambda bound to the lifetime of this.
co_await BoundAsyncLambda();
```

Example: 12_AsyncLambdas

Compiler gotchas and other considerations

Exception support

- Clang has a bug where capturing an exception and rethrowing it across coroutines results in a crash because it doesn't generate the correct code to handle `std::exception_ptr`
 - <https://github.com/llvm/llvm-project/issues/182584>
- So `#if` guard around your exception capture-and-throw logic for Clang, even if it would otherwise be enabled for the target

Notes about memory allocation - coroutine state

- C++20 coroutines are stackless
- A coroutine will allocate its state object via operator new
- There are ways to override operator new from the promise type
- *Unfortunately*, you can't really wire this up to any kind of "async stack"
- Even if we could get some kind of "async stack" from parent context, no way to guarantee the coroutine is the next thing awaited (or even awaited at all).

```
static TTask<void> VoidCoroutine()
{
    co_await FDelayOnAnotherThread{};

    co_return;
}

std::vector<TTask<void>> ExampleListOfTasks;

static TTask<void> ScheduleTask()
{
    TTask<void> Task = VoidCoroutine();

    ExampleListOfTasks.push_back(Task);

    co_return;
}
```

Notes about memory allocation - coroutine state

- Compilers are supposed to be able to move the allocation to the parent coroutine frame when possible: (*HALO - Heap Allocation eLision Optimization*)

The call to `operator new` can be optimized out (even if custom allocator is used) if

- The lifetime of the coroutine state is strictly nested within the lifetime of the caller, and
- the size of coroutine state is known at the call site.

- In practice, this doesn't often happen 😞
 - MSVC is very bad at it
 - Clang is bad at it - can make it somewhat better by adding `[[clang::coro_await_elidable]]` attribute to your TTask on very new Clang versions

`-std=c++20 -O3 -Rpass-missed=coro`

Notes about memory allocation - task state

- I would like to move task state into coroutine promise to eliminate an additional heap allocation.
 - *Note:* In our actual task library implementation, we already eliminate the task state heap allocation if a coroutine never suspends; we put the value directly into the TTask<> that is yet to return to the caller.
- To do this, you need to make the coroutine promise ref-count itself.
- Work is ongoing... some benchmarks are currently faster, some are currently slower.
- A lot of our TTask usage is with virtual calls for which HALO can never remove the coroutine promise heap allocation, so we are unlikely to ever avoid heap allocations entirely.

Things I don't have time to cover

- Thread policies
 - Automatically making sure a coroutine runs on a desired thread, and returning to the correct thread when the continuation happens
- FExecuteOnCancellation
 - `co_await` to register an async lambda that should run if a coroutine is cancelled before returning
 - Kind of a "finally" block to run for coroutines since cancellation can be imposed from lifetime checks, or requested by an external caller at any time
 - Important if you're implementing transactional code in coroutines

Things I don't have time to cover

- Exceptions
 - Necessary if you are compiling with exceptions enabled, otherwise debuggers will not breakpoint where they otherwise would
 - `unhandled_exception()` on the promise object, call `std::current_exception()` and stash the `exception_ptr` in the task state
 - `await_resume` rethrow the exception with `rethrow_exception`

Questions?

Important Links

- Slides: junerhodes.au/history
- C++20 coroutines spec: en.cppreference.com/cpp/language/coroutines
- Examples/demo: github.com/hach-que/GCAP2026Examples

Social

- Mastodon: mastodon.social/@hq